

LLM-on-the-Palm: Mobile LLM Inference with PIM-Enhanced NAND Flash Memory

Hyunjin Kim*, Sanghyeok Han*, Jae-Joon Kim
Seoul National University, Seoul, Republic of Korea
{hyunjin.kim, hans778, kimjaejuon}@snu.ac.kr

Abstract—Large Language Model (LLM) inference on edge devices is crucial for democratizing AI and addressing privacy and security concerns associated with cloud services. However, the large parameter sizes of LLMs pose significant challenges in deployment on resource-constrained devices, affecting user experience. Mobile DRAM typically cannot accommodate these parameters, necessitating the use of NAND flash memory, but it has considerably slower access speeds. To overcome the limitations, we propose LLM-on-the-Palm, an LLM inference system for smartphones that leverages Processing-in-Memory (PIM)-enhanced NAND flash memory to enable fast inference of billion-parameter LLMs with minimal hardware modifications. Our approach requires only a single multiply-accumulate (MAC) unit per plane in a NAND flash memory - 256 MAC units for 256 GB SSD - without compromising cell density. Simulation results show that our design achieves ~ 100 ms per token generation on a 6.7B model under specifications similar to an iPhone-15.

I. INTRODUCTION

LLM inference on edge devices is garnering attention due to the concerns for privacy and security issues [1], [2]. However, LLM inference requires large system resources due to its large number of parameters. Since DRAM capacity in mobile devices is not large enough to store all parameters, they are typically brought from NAND flash memory. However, NAND flash memory is much slower than DRAM, resulting in unacceptable performance. Fig. 1(a) shows this challenge, in case of specifications close to that of an iPhone-15. For a 16-bit 6.7B model, at least 13 GB of memory space is needed. Recently, LLM-in-a-flash [3] introduced optimization techniques to accelerate LLM in laptops using NAND flash memory, by pruning weight parameters and caching parameters in DRAM for reuse. However, it has several limitations. First, the pruning technique often involves accuracy drop problem, which hinders application of various LLMs. Second, it involves memory management overhead to support DRAM caching. Most importantly, it still occupies large portion of DRAM, thus not scalable to larger models and hard to apply to more resource-restricted devices such as mobile phones.

We aim to run LLM inference on smartphones targeting the time per output token (TPOT) of ~ 100 ms which is known to be faster than a person can typically read, without modifying pre-trained weights [4]. To achieve this goal, we propose *LLM-on-the-Palm*, a mobile system for rapid LLM inference with Processing-In-Memory (PIM)-enhanced NAND flash memory. Our system exploits Flash-PIM [5]–[7], which embeds computing units inside NAND flash memory to reduce off-chip transfer of weights. The elimination of loading a huge amount of weight parameters from storage to NPU significantly reduces the data transfer latency.

One of our primary design goals is to minimize the hardware overhead to enable PIM capability within NAND flash memory. Cambricon-LLM [6] recently exploited Flash-PIM for LLM inference at the edge computing environment. However, it is challenging to adopt Cambricon-LLM style design in smartphone systems due to

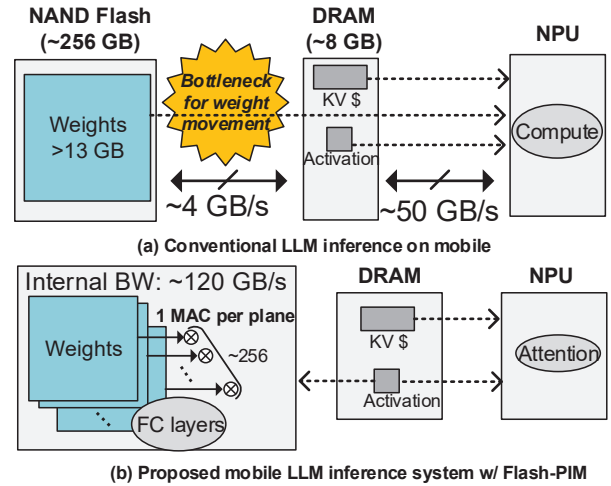


Fig. 1: Comparison of existing mobile LLM inference scenario and our proposed mobile LLM inference based on Flash-PIM.

its limited plane utilization, which stems from significant buffer area overhead. This limitation degrades cell read throughput and prevents the system from meeting Service Level Objective (SLO) for smartphones. Furthermore, Cambricon-LLM requires a dedicated neural processing unit (NPU) to be integrated into the controller die, which incurs significant modification of the existing smartphone system and high I/O energy consumption when transferring weights to the controller die. In contrast, our design employs only a single Multiply-Accumulate (MAC) unit per NAND flash plane, minimizing its overhead for lightweight, power-constrained devices.

To design LLM-on-the-Palm, we analyze the characteristics of LLM inference when executed on Flash-PIM architectures. LLM inference involves various computation kernels, each with distinct computational and memory access patterns that affect the performance of Flash-PIM. We conducted both qualitative and quantitative analysis of the behavior of NAND flash memory during LLM inference and designed the optimal inference scheme on mobile systems.

In mobile LLM system design, minimizing cost and overhead is critical. To address the issue, we implement a performance- and area-efficient Flash-PIM architecture for LLM inference. Through a detailed analysis of NAND flash memory cell latency and compute latency, we discovered that incorporating just a single MAC unit per plane is sufficient to fully exploit the inherent plane-level parallelism in NAND flash memory, incurring negligible impact on area and energy.

Fig.1(b) illustrates an exemplary LLM inference system configuration based on our proposed scheme, assuming iPhone 15-level specifications. We offload prefill to NPU since it requires substantial

*These authors contributed equally to this work.

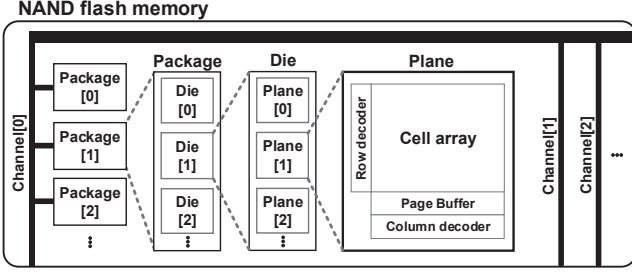


Fig. 2: NAND flash memory architecture.

compute resources, so the figure focuses on the decoding process. In this design, a MAC unit is placed near each NAND flash plane to perform multiplication between the weights from the plane and input vectors supplied from DRAM. Flash-PIM provides much higher bandwidth than off-chip bandwidth, even with only ~ 256 MACs for 256 GB storage because its plane-parallel operation enables much higher internal bandwidth compared to the external bandwidth. To optimize performance, Flash-PIM and NPU in our system are assigned different layers in each transformer decoder block. Although attention mechanisms are memory-bound, we observed that they incur significant overhead when executed on NAND flash memory due to the auto-regressive nature of LLM decoding. Therefore, as shown in Fig. 1(b), we store all activations, including the key-value (KV) cache, on DRAM and offload attention computations to NPU. Flash-PIM is dedicated to processing fully-connected (FC) layers, which involve multiplication of large weight parameters and activations.

In our simulations on a mobile platform with iPhone 15-level specifications (using the NAND flash specifications from Galaxy S24, as iPhone 15 NAND specs are not publicly available), our system achieves 100 $\sim$ 120 ms of time-per-output-token (TPOT) for a 6.7B model, which is $\sim 30\times$ faster than the conventional configuration based on Flash memory. Furthermore, by integrating model pruning techniques [3] in our scheme, we can support larger models (e.g. 13B and 30B) with 155 and 356 ms TPOT, respectively.

II. PRELIMINARIES

A. NAND flash memory

NAND flash memory is a non-volatile storage that offers high density and low cost per bit, making it widely used in solid-state drives (SSD). To enhance parallelism and storage efficiency, NAND flash is organized in a hierarchical structure as shown in Fig. 2. It comprises multiple channels, each connected to several NAND flash packages. A package contains multiple dies, and each die is further divided into multiple planes. A plane consists of a cell array along

with peripheral circuits such as a row decoder, a column decoder, and a page buffer. The row and column decoder select target cells within the array, while the page buffer temporarily stores data either being read from or to be written to the cell array.

During read and write, data movement between the cell array and the page buffer incurs substantial latency overhead, necessitating parallel operation across multiple planes to achieve high throughput [8]–[10]. However, channel bandwidth limits the throughput of such multi-plane operation, as multiple planes share a channel. To enhance throughput by better utilizing plane-level parallelism, recent studies have proposed to integrate computing unit within NAND flash die, which will be detailed in Section II-C.

B. LLM inference on mobile platforms: DRAM capacity and Flash memory bandwidth issues

There is a growing demand in industry for running LLM inference on mobile platforms due to the privacy and security issues in cloud services [1], [2]. However, performing LLM inference on mobile platforms remains highly challenging compared to other AI models such as convolutional neural networks (CNNs). While CNNs benefit from high parallelism by simultaneously processing numerous pixels, LLMs are inherently more memory-bound due to their auto-regressive nature. More importantly, a critical limitation lies in the mobile DRAM capacity, which is insufficient to load the vast number of parameters required by LLMs. Fig. 3(a) shows the capacity of DRAM in recently released smartphone models alongside various LLM model parameter sizes. None of the devices can fully accommodate model parameters ranging from 6.7B to 30B. Consequently, model parameters should be brought from the storage, typically NAND flash, at runtime. Fig. 3(b) shows that even the smallest NAND flash capacity (128 GB) commonly found in smartphones can store a 30B model. The challenge, however, still lies in bandwidth. While LPDDR5 offers around 50 GB/s, typical smartphone SSDs (e.g., 256 GB) provide only about 4 GB/s. The large bandwidth gap between DRAM and NAND flash results in significant data transfer delays from NAND flash, which dominates the end-to-end latency. It makes LLM inference unacceptably slow for a satisfactory user experience.

C. Related work on edge LLM inference using Flash memory

Recently, *LLM-in-a-flash* [3] demonstrated an approach to reduce LLM inference latency by storing model parameters in NAND flash memory and reducing data transfer from NAND flash by predicting zero activation values and selectively reading only the necessary weights. It also optimizes DRAM usage under limited capacity by caching a subset of model parameters in DRAM and reusing them across multiple token generations.

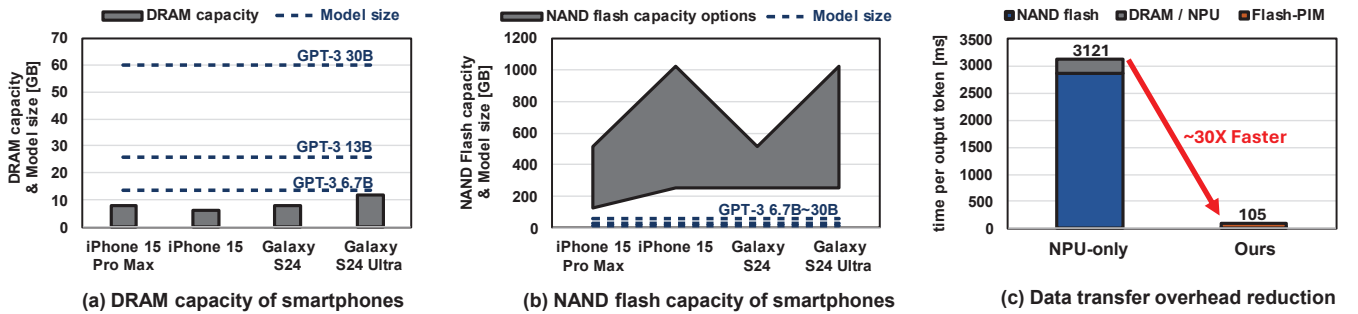


Fig. 3: Analysis of (a) DRAM and (b) NAND flash capacity of smartphones with LLM model sizes and (c) data transfer overhead in NAND flash for GPT-3 6.7B inference (128 input tokens and 128 output tokens).

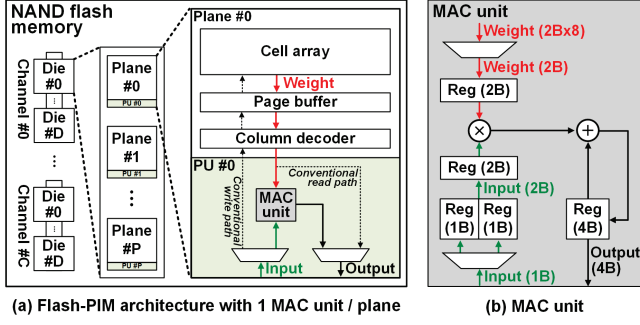


Fig. 4: (a) Proposed Flash-PIM architecture, where the green box represents an additional processing unit integrated into conventional NAND flash, and (b) its area-efficient MAC unit.

However, this approach has several limitations. First, its reliance on pruning techniques can lead to noticeable accuracy degradation, which restricts its applicability across diverse LLM architectures [11], [12]. Furthermore, it assumes activation sparsity by using ReLU [13] activation function, making it less compatible with models that use GeLU [14] functions [15]. Second, the method incurs memory management overhead to support DRAM caching. It requires rearranging memory space for each token generation to minimize DRAM cache size, which increases DRAM access latency. More importantly, it still consumes a large portion of DRAM for DRAM caching. For a 6.7B model, it uses 6.5 GB of 14 GB capacity of DRAM, assuming only half of the DRAM is available for LLM inference since the other half is needed for other smartphone applications. Furthermore, heavy DRAM usage limits scalability in terms of both model size and context length, the latter of which increases the KV cache size in DRAM. In summary, the high DRAM overhead hinders the applicability of LLM-in-a-Flash to mobile devices and longer contexts.

A promising approach to address the data bandwidth issue is using Flash-PIM [5]–[7], which reduces data transfer overhead by integrating computing units within NAND flash. Instead of transferring large raw data from NAND flash to off-chip, Flash-PIM transfers the smaller computation results by processing the data inside NAND flash. As shown in Fig. 3(c), our Flash-PIM architecture significantly reduces off-chip data transfer, achieving more than a 10x speedup and reaching close to reasonable service time. Similar to our work, Cambricon-LLM [6] employs Flash-PIM for LLM inference. However, it faces several limitations. First, the large input buffer required for its Flash-PIM computing units prevents them from being implemented in every plane. As a result, a single computing unit must be shared across multiple planes, leading to plane under-utilization and lower throughput. Second, Cambricon-LLM requires additional NPU integrated into the NAND flash controller to co-process weights stored in NAND flash alongside the Flash-PIM computing units to compensate for the low plane utilization. In contrast, our proposed architecture minimizes buffer overhead by pipelining MAC computation with input transfers. This reduction in buffer overhead allows the placement of a PIM unit in each plane, enabling multiple planes within a die to execute PIM operations in parallel, significantly increasing throughput. Furthermore, our architecture does not need an extra NPU due to the abundance of PIM units.

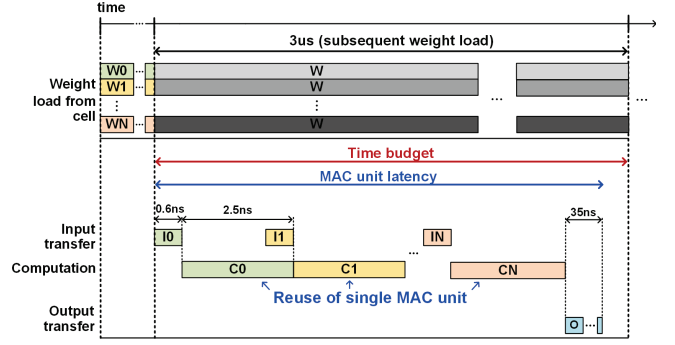


Fig. 5: Optimized Flash-PIM operation with a single MAC unit.

III. OPTIMIZING FLASH-PIM ARCHITECTURE FOR LLM

A. Overall architecture

We design a Flash-PIM architecture optimized for vector-matrix multiplication (GEMV) operations in LLM inference. Our architecture enhances performance by pipelining MAC operations with input transfers from the host and subsequent weight fetches from the cell array. As shown in Fig. 4, the architecture integrates a processing unit (PU) adjacent to the column decoder in each NAND flash plane. Each PU includes a single MAC unit consisting of a 16-bit floating-point multiplier and a 32-bit floating-point adder, a demultiplexer that routes input data either to the cell array for conventional memory writes or to the MAC unit for PIM computations, and a multiplexer to select the output from either the cell array for conventional memory read or from the MAC unit. The MAC unit performs GEMV operations between the external input vectors and the weight values read from the cell array. Since GEMV involves computing the product of a single input vector and a 2-dimensional weight matrix, the same input data is shared across multiple planes. To support this, the input vector is broadcast through channels to all planes.

B. MAC unit optimization

By integrating only one MAC unit per plane, our Flash-PIM architecture significantly reduces power and area consumption. Due to the highly memory-bound nature of LLM inference, this lightweight computing unit does not compromise overall performance. Despite having only 256 MAC units in total for a 256GB NAND flash [10], our design matches the performance of designs that use substantially more MAC units.

Fig. 5 shows why a single MAC unit per plane in Flash-PIM is sufficient for LLM inference. In this figure, we assume MAC units operating at 400 MHz integrated into a 256GB storage made up of a low-latency NAND flash, Z-NAND [10]. During each MAC operation, the subsequent row of weights (i.e., a page) is read from the cell array and stored in the page buffer, which takes approximately $3\mu s$. We use the $3\mu s$ cell read latency as the computation window for the MAC unit, ensuring no performance loss as long as entire MAC operations are completed within this time frame.

Each MAC operation cycle includes input transfer, computation, and output transfer phases. In each cycle, the MAC unit processes 2B of input data and 2B of weight data. Input data transfer requires 0.6ns per 2B transfer at a channel bandwidth of 3.2GB/s [16], which is significantly shorter than a MAC computation time of 2.5ns. Based on the observation, input transfers are pipelined with MAC computations to optimize performance. Using a conventional 1B-wide channel, the next input data is preloaded into a 1B register (Fig. 4(b)). This pipelining strategy reduces the overall computation latency and

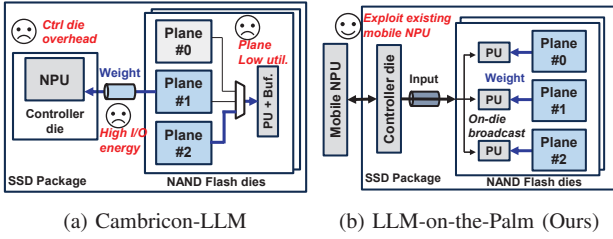


Fig. 6: System difference between Cambricon-LLM and ours.

eliminates the need for a large input buffer, significantly reducing area overhead. During computation, the MAC unit iteratively processes 1024 weight elements that were stored in the Z-NAND page buffer, completing the operation in 1024 cycles ($2.5\mu s$). After these computations, each plane outputs a 4B result to an external device through the channel, consuming 35ns. Since the $\sim 2.5\mu s$ of overall latency of the 1024 MAC operations using a single MAC unit is shorter than the cell read latency, $3\mu s$, each MAC unit completes its task before the next page is ready. Consequently, throughput is maximized and effectively limited only by the cell-read latency. As a result, our proposed Flash-PIM architecture can achieve high performance with just a single MAC unit per plane. Although cell read latency is relatively long, parallel cell reads across multiple planes provide high aggregate throughput (i.e., 117GB/s for our configuration), enabling efficient and fast LLM inference.

C. ONFI command for Flash-PIM

We design a control scheme for the Flash-PIM architecture within a conventional NAND flash interface protocol, ONFI [17]. The controller transfers control signals to NAND flash through ONFI pins such as DQ (i.e., data pin) and CE_# (i.e., chip-enable # pin). To transfer input for GEMV, we utilize the standard write data transfer mechanism, but extend it to broadcast input data across multiple NAND flash dies. While DQ pins are connected to the shared data bus in a channel, the controller transfers input data to the shared bus and set CE_# to 0 for target NAND flash dies for input broadcast. While conventional NAND flash selects a single NAND flash die per channel, our design can select multiple NAND flash dies as CE_# of each die is controlled by separate signal lines. Additionally, to distinguish Flash-PIM operation with conventional NAND flash read, we assign a new opcode for Flash-PIM (36h) among vendor specific opcodes of ONFI.

D. System efficiency of the proposed scheme on smartphones

Compared to the prior Flash-PIM based LLM inference architecture, Cambricon-LLM [6], the proposed LLM-on-the-Palm offers three key advantages for performance, area, and energy as illustrated in Fig. 6. Cambricon-LLM adopts chiplet technology to implement NPU on the controller die, incorporating components such as a systolic array, SRAM buffers and special function units, etc. The NPU performs near-memory processing by transferring weights from NAND flash dies to the controller die over chiplet interconnects.

(1) In contrast, LLM-on-the-Palm leverages existing mobile NPUs without requiring chiplet-based interconnects for high-bandwidth weight transfer. The NPU is used exclusively for attention operations, which do not require model weights. All computations involving weights are processed by Flash-PIM units within NAND flash dies, eliminating external weight transfers, as detailed in Section IV.

(2) Despite the lightweight architecture, LLM-on-the-Palm achieves higher throughput than Cambricon-LLM through improved plane utilization. While Cambricon-LLM's on-die processing units

require large input buffers, forcing multiple planes to share a single unit, our design integrates a compact processing unit into every plane. Additionally, by avoiding weight transfers to the NPU and broadcasting input data to all dies and planes via shared channels, LLM-on-the-Palm reduces channel I/O traffic. This enables pipelined input transfer and MAC computation with minimal register usage, as explained in Section III-B, allowing for smaller area footprint per processing unit and full-plane deployment.

(3) Finally, LLM-on-the-Palm achieves lower energy consumption by reducing channel I/O traffic. In our design, weight remain local within the NAND flash dies, and only input vectors are transferred through I/O channel. In addition, channel-level and on-die broadcasting further reduce the I/O burden. In contrast, Cambricon-LLM incurs higher energy costs due to frequent and large-scale weight transfers from NAND flash to the NPU.

IV. LLM PROFILING AND MAPPING FOR LLM-ON-THE-PALM

A. Prefill vs. Decoding

LLM inference consists of operations with diverse computational characteristics, each impacting performance of Flash-PIM architecture differently. LLM inference can be divided into two stages: prefill and decoding, which have different operational intensities. The prefill stage processes input context to generate the first output token. This stage typically involves computing tens to thousands of tokens in parallel and is implemented as a matrix-matrix multiplication (GEMM) kernel. GEMM is compute-bound and highly optimized for NPUs, which reuse weights stored in on-chip SRAM caches to hide memory access latency. Despite the bandwidth limitations of NAND flash, the NPU's larger compute capacity leads to a lower time-to-first-token (TTFT) compared to Flash-PIM. Based on the observation, we offload the prefill task to the NPU. Detailed performance comparison between NPU and Flash-PIM for prefill tasks will be shown in Section IV-B.

Unlike the prefill, decoding stage is memory-bound due to its auto-regressive nature, and tokens are generated sequentially. As a result, the decoding stage relies on vector-matrix multiplication (GEMV) kernels, which impose memory bottlenecks not only on NAND flash but also on DRAM. To relax the memory bottleneck, batching techniques [18]–[20] have been proposed, which handle multiple requests simultaneously to convert GEMV operations to GEMM operations and increase token-level parallelism. However, since LLM inference on smartphones typically involves single-user scenarios, batching techniques that are commonly used in data centers are generally not applicable in such cases. Therefore, our target workload for decoding stage is the single-batch case, where only GEMV operations are performed. Since GEMV is memory-bound, it presents a strong opportunity for acceleration using Flash-PIM, which is designed to alleviate memory access bottlenecks.

B. Selective layer-wise mapping strategy for Flash-PIM and NPU

While Flash-PIM is effective for memory-bound operations, the transformer decoder block contains diverse types of layers that impact Flash-PIM performance differently, as illustrated in Fig. 7(a). These layers can be categorized into two types based on their computation patterns: *weight-activation multiplication* and *activation-activation multiplication* layers. Fully Connected (FC) layers such as Feed Forward Network (FFN) and the QKV generation and projection layers in the attention module are based on weight-activation multiplications. These layers involve the multiplication of input feature vectors with pre-trained weight matrices. Since weights are static and stored in NAND flash prior to runtime, Flash-PIM achieves a

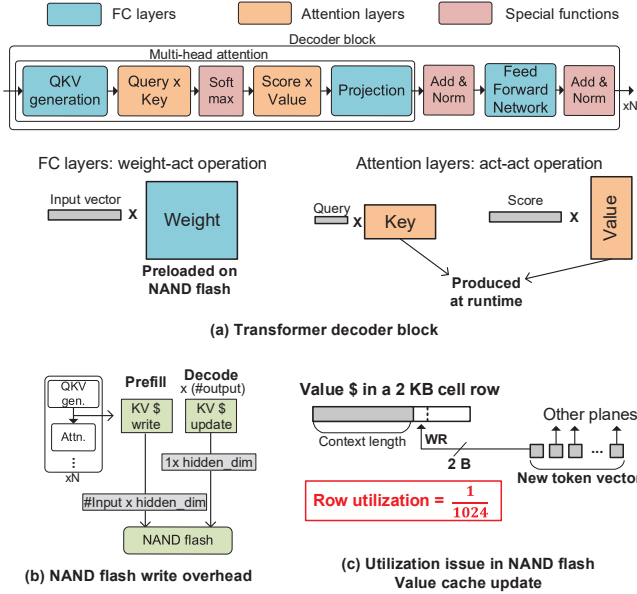


Fig. 7: Analysis of each operation in LLM decoding on Flash-PIM. Activation-activation multiplication incurs NAND-Flash write overhead, especially in Score $\times$ Value (SxV) operations.

significant speedup by eliminating the need to frequently transfer large volumes of weights. As described in Fig.3(c), it takes ~ 3 seconds to transfer 6.7B parameters from NAND flash to NPU, which dominates the entire runtime. In this case, the abundance of compute resources in NPU does not help to reduce the runtime. In contrast, our Flash-PIM reduces the transfer latency to less than 100 ms, leveraging the high internal bandwidth enabled by parallel reads from NAND flash planes to their dedicated PUs. Note that we can significantly reduce the overall execution time by offloading only the FC layers to Flash-PIM, which accounts for up to 99.5% of decoding latency when executed on NPU.

On the other hand, attention operations, specifically Query $\times$ Key (QxK) and Score $\times$ Value (SxV), are activation-activation multiplications and are assigned to the NPU and DRAM. This is due to their high overhead when executed with NAND flash memory as the input vector is multiplied by activation data, either the Key or Value matrix from the KV cache generated at runtime. In such cases, KV cache produced from QKV generation layer should be written to NAND flash when the prefill completes and should be updated for each token generation as described in Fig. 7 (b), causing extra overhead to write NAND flash.

Moreover, the auto-regressive nature of decoding increases the KV cache update overhead. Unlike the prefill, the KV cache size grows incrementally with each token, and only a single token vector of the hidden dimension (e.g. 4096 for GPT-3-6.7B) can be appended at a time. NAND flash throughput deteriorates as the write data size becomes smaller as shown in Fig. 8. For example, in GPT-3-6.7B model, NAND flash throughput drops more than $10\times$ from the maximum I/O throughput. For SxV, the NAND flash write overhead becomes particularly severe. Since SxV processes dot products between the previous token elements, sequential token elements of Value matrix should be stored in the same row in a NAND flash cell array to be read simultaneously as shown in Fig. 7(c). However, when appending a new token, only one 16-bit data element can be added to a cell row of 2 KB while other elements should be distributed across multiple planes. This leads to decrease in row utilization by up to $1024\times$,

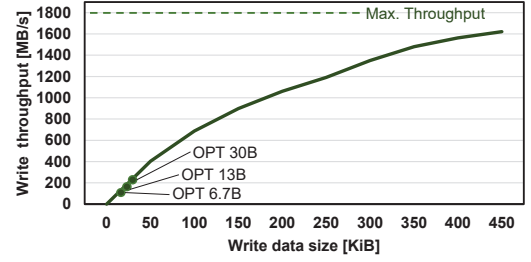


Fig. 8: Dependence of NAND flash write throughput on the write data size. The write throughput increases as the data size increases but KV cache update size in LLM decoding is significantly lower than the desired data size for the maximum throughput.

TABLE I: System configuration

NAND flash	Interface	Host interface: UFS 4.0, Flash interface: ONFI 5.2
	Capacity	SSD capacity = 56GB, Die capacity = 8GB
	Die configuration	4 dies/channel, 8 planes/die, tR=3 μ s
	Bandwidth	Channel: 3.2GB/s, Host interface: 4.2GB/s
	Supply	VCC=3.3V, ICC1=70mA
NPU	Apple Neural Engine Gen6	
DRAM	LPDDR5 6GB	

significantly increasing latency.

Our profiling results in Fig. 9 support this analysis. As shown in Fig. 9a, Flash-PIM is significantly faster than NPU in FC layers. However, in attention operations shown in Fig. 9b and Fig. 9c, Flash-PIM becomes slower than NPU due to significant overhead to write NAND-Flash. In particular, SxV performance is dominated by the high NAND flash write latency due to the poor row utilization (Fig. 9c).

Based on the analysis, we assign the FC layers to Flash-PIM and QKV multiplication to NPU, as depicted in Fig.1. An exception is the Query $\times$ Key multiplication for long input sequences. When the context length is long, the benefits of alleviating memory bottleneck with Flash-PIM outweigh the NAND flash write overhead (Fig. 9b). The runtime breakdown in Fig. 9d shows that longer input context length increases the proportion of attention layers, which underlines the importance to accelerate attention by using Flash-PIM. In addition, as shown in the KV cache size in Fig. 9d, KV cache size can exceed the DRAM capacity as the input length becomes larger. By moving the Key cache to the NAND flash memory, which is the half of KV cache, we can save the DRAM capacity and handle $2\times$ longer context requests. Therefore, we can further optimize the Flash-PIM usage by selectively offloading Query $\times$ Key task to NPU or Flash-PIM. For a short input context, NPU handles all attention operations. However, when the request has a much longer context (e.g., 4096), Flash-PIM handles a part of the attention layers; Query $\times$ Key multiplication, saving DRAM space and enhancing performance.

V. EVALUATION

A. Evaluation setup

In this section, we compare the proposed architecture with the conventional LLM inference architecture using NPU (referred to as NPU-only in this paper) and Cambricon-LLM [6]. Our evaluation includes single-batch inferences on smartphones for three variants of GPT-3 [15]: 6.7B, 13B and 30B. In this setup, weights are stored in NAND flash due to their large size, while KV caches reside in DRAM. Additionally, we analyze the area and power overhead of Flash-PIM and its sensitivity to different NAND flash types and capacities.

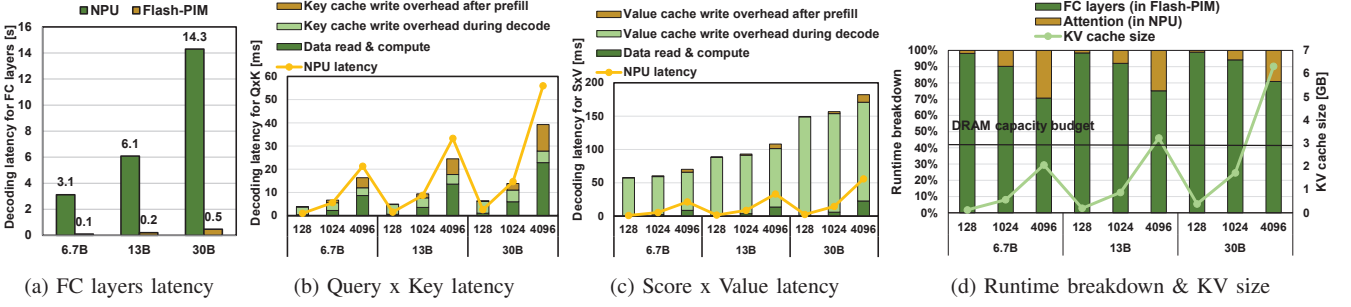


Fig. 9: Profiling results for decoding stage of GPT-3 [15] on NPU and Flash-PIM with different parameter sizes (6.7B, 13B, 30B) and different context lengths (128, 1024, 4096) for generating 128 tokens.

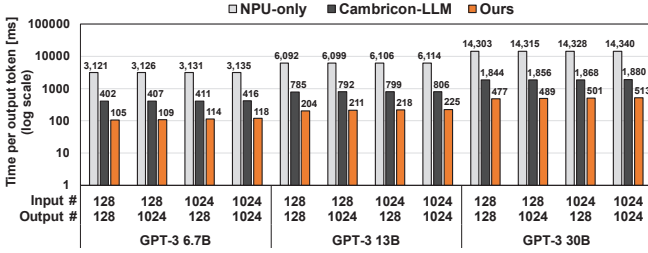


Fig. 10: Time per output token of the conventional architecture (NPU-only), Cambricon-LLM, and the proposed architecture (Ours).

Table I shows the system configuration used for evaluation. For NAND flash configuration, we use 256GB UFS 4.0 [21] mobile storage with Z-NAND [10], which provides cell latency (tR) of 3us and channel bandwidth of 3.2GB/s. We use single level cell, which does not require error correction code (ECC) computation. Detailed discussion on ECC will be provided in Section VI-A. The NAND flash operates at VCC=3.3V and ICC1=70mA for multi-plane read. DRAM and NPU configurations follow iPhone 15 specifications, using LPDDR5 6GB and Apple Neural Engine Gen6, respectively. We simulate NAND flash latency with MQSim [22] and DRAM latency with DRAMSim [23]. Power consumption for NAND flash is evaluated using the Micron Parallel NAND System Power Calculator. MAC units are synthesized in 28nm CMOS technology with Synopsys Design Compiler [24] to estimate area and power consumption.

B. Decoding latency

Fig. 10 compares the time per output token (TPOT) between the NPU-only baseline and the proposed architecture. TPOT represents the decoding latency required to generate a single output token. For the evaluation, we use four different input-output token count pairs. The average TPOTs for the proposed architecture are 112ms, 215ms, and 495ms for GPT-3 models with 6.7B, 13B, and 30B parameters, respectively. This results in a geometric mean speedup of $28\times/3.7\times$ over the NPU-only/Cambricon-LLM, respectively. As expected, longer input contexts increase TPOT as they increase the computation overhead in attention layers due to the larger KV cache size. In addition, larger models require reading more weights from NAND flash, further increasing TPOT.

To further reduce TPOT, we apply LLM-in-a-flash [3] techniques discussed in Section II-C. While LLM-in-a-flash provides reasonable TPOT for NPU-only on edge devices with larger DRAM capacities such as laptops, it does not provide sufficient speedup on smartphones due to the limited DRAM capacity. In contrast, by combining the proposed architecture with LLM-in-a-flash techniques, we can

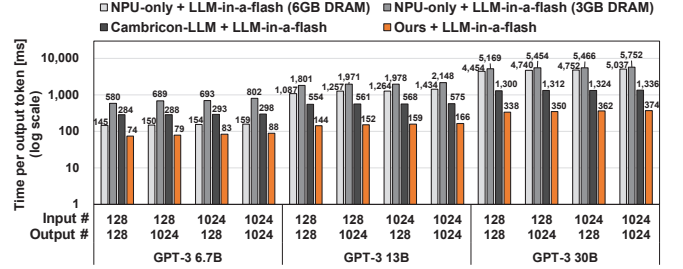


Fig. 11: Time per output token when LLM-in-a-flash [3] techniques are applied.

achieve substantial speedups and meet the reasonable user experience level even for larger models such as 13B or 30B on smartphones.

Fig. 11 shows evaluation results of TPOT after applying LLM-in-a-flash techniques. While NPU-only baseline benefits from the DRAM-based weight caching of LLM-in-a-flash, Flash-PIM-based architectures cannot benefit since computation is performed directly within NAND flash and weights are not transferred to DRAM. Since the effectiveness of DRAM weight caching depends on the available DRAM capacity for storing the weights, we consider two scenarios for the NPU-only baseline: (1) a full DRAM capacity (6GB) is allocated to LLM inference, (2) only half the capacity (3GB) is available. The first case with 6GB DRAM represents an ideal scenario where the entire DRAM is dedicated to LLM, while the second case with 3GB DRAM space reflects a more realistic scenario where the half of the DRAM is used by other applications, as in the LLM-in-a-flash experiments [3]. Under these scenarios, the proposed architecture achieves reduced average TPOTs of 81ms, 155ms, and 356ms for 6.7B, 13B, and 30B, respectively. This corresponds to geometric mean speedups of $5.9\times$ and $12\times$ over the NPU-only baseline with 6GB and 3GB available DRAM, respectively. Note that the performance of the NPU-only baseline is still limited even with LLM-in-a-flash DRAM caching, due to DRAM capacity constraints. As model size and context length increases, or the available DRAM capacity decreases, the effectiveness of LLM-in-a-flash techniques diminishes for the NPU-only baseline. In contrast, the performance of the proposed architecture does not depend on the available DRAM capacity as it leverages Flash-PIM rather than DRAM weight caching. The proposed architecture also achieves a $3.6\times$ speedup over Cambricon-LLM when LLM-in-a-flash techniques are applied, thanks to its more efficient design. Note that Cambricon-LLM also does not utilize DRAM for weight caching in this comparison.

C. Prefill latency

While the proposed architecture employs both Flash-PIM and NPU for decoding stage, which dominates the end-to-end latency, it relies

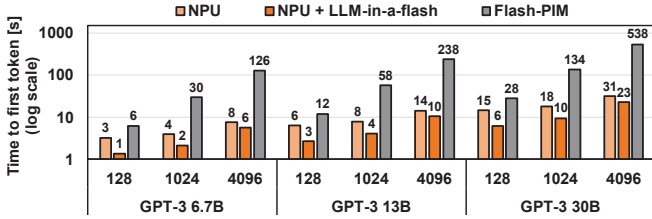


Fig. 12: Prefill latency of NPU and Flash-PIM. We use NPU for prefill stage.

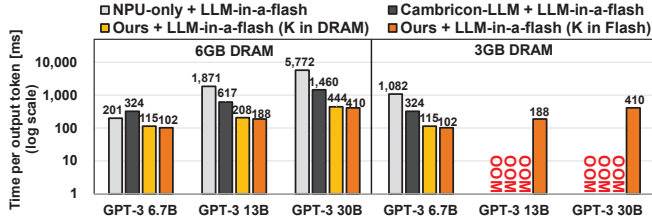


Fig. 13: Latency for long sequences in (a) decoding stage and (b) prefill stage. We use NPU for prefill stage.

solely on NPU for prefill stage. As shown in Fig. 12, NPU achieves lower time to first token (TTFT), defined as the latency of prefill stage to generate the first output token, compared to Flash-PIM. As prefill stage requires GEMM computations on large amount of inputs, which require high-throughput input transfer, Flash-PIM suffers from high latency to transfer input through the low channel bandwidth (i.e., 3.2GB/s). Given the importance of TTFT for user experience, we can apply LLM-in-a-flash techniques to reduce TTFT. NPU-only with LLM-in-a-flash achieves average TTFT of 1.5s/3.5s/8s, for GPT-3 6.7B/13B/30B, respectively.

D. Long sequence latency

To evaluate TPOT for long sequences, we simulated decoding of GPT-3 variants with 4096 input tokens and 128 output tokens, as shown in Fig. 13. We applied LLM-in-a-flash technique for all architectures. As demonstrated in Section IV-B with Fig. 9b, storing Key cache in NAND flash reduces the amount of DRAM usage and prevents out-of-memory (OOM) in DRAM during long sequence decoding, while also improving performance. With 6GB of available DRAM, our architecture achieves speedup of $2.0\times/10\times/14\times$ over the NPU-only baseline for GPT-3 6.7B/13B/30B models, respectively. Compared to Cambricon-LLM under the same DRAM constraint, our architecture achieves $3.2\times/3.3\times/3.6\times$ for GPT-3 6.7B/13B/30B models, respectively. Note that the NPU-only baseline slightly outperforms Cambricon-LLM for GPT-3 6.7B when 6GB of DRAM is available, as the NPU-only greatly benefits from the LLM-in-a-flash technique by caching a large portion of weights due to small model size and relatively large available DRAM capacity. With 3GB of available DRAM, for GPT-3 6.7B, our architecture achieves speedup of $11\times/3.2\times$ over NPU-only/Cambricon-LLM, respectively. For GPT-3 13B and GPT-3 30B, our architecture achieves TPOTs of 129ms and 272ms, respectively. However, NPU-only and Cambricon-LLM encounter DRAM out-of-memory errors, which our architecture avoids by offloading Key cache to NAND flash.

E. Area overhead

Our architecture introduces relatively small area overhead to implement processing units in Flash-PIM, enabling the deployment of one processing unit per NAND flash plane. This high plane-level parallelism significantly improves throughput. In contrast, Cambricon-

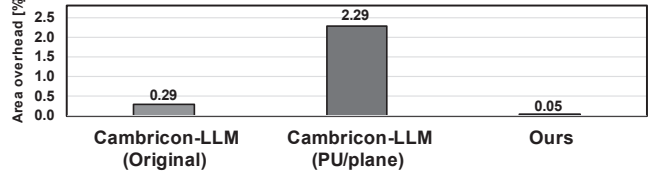


Fig. 14: Area overhead of processing units relative to flash die.

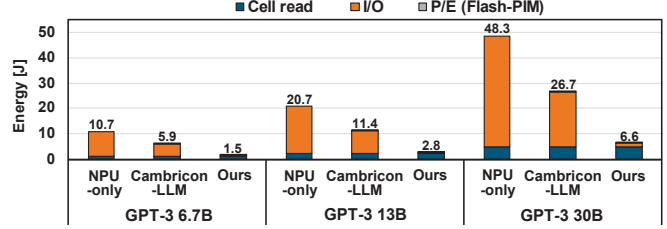


Fig. 15: Comparison of energy consumption.

LLM does not fully exploit plane-level parallelism, as multiple planes within a NAND flash die share a single processing unit, resulting in lower throughput. Fig. 14 shows the area overhead relative to NAND flash dies. The Cambricon-LLM paper [6] reports that placing a processing unit in every plane for Cambricon-LLM was not feasible as it incurs 2.4% area overhead. Our evaluation confirms the finding. In our study, placing processing unit in each plane for Cambricon-LLM, labeled as Cambricon-LLM (PU/plane) in Fig. 14, showed 2.3% area overhead. In contrast, our architecture drastically reduces buffer size as explained in Section III-B, achieving only 0.05% area overhead. This demonstrates the feasibility of placing a processing unit for each plane in our design.

F. Energy consumption

Fig. 15 shows the energy consumption in NAND flash during each output token generation. The proposed architecture reduces energy consumption of NAND flash by $8.3\times/4.5\times$ compared to NPU-only/Cambricon-LLM, respectively. In mobile systems with limited battery capacity, low energy consumption is critical. For the NPU-only baseline which consumes the highest energy, I/O accounts for 89.5% of total energy to transfer weight to NPU from NAND flash. Cambricon-LLM reduces I/O energy by reducing the amount of data transfer using Flash-PIM, but I/O energy still constitutes 80.7% of total energy as it transfers a portion of weights to the external NPU. In contrast, the proposed architecture fully utilizes Flash-PIM, reducing I/O energy to 21.8% of the total energy. All three architectures consume the same energy for flash cell reads, as the same number of data needs to be read from Flash array. The proposed design incurs a small additional energy (1.3%) for additional program/erase (P/E) for Flash-PIM operations. However, the overhead is relatively small and the proposed design achieves the lowest total energy consumption.

G. Sensitivity analysis

A sensitivity analysis on latency, averaged across the token numbers used in evaluations of Section V-B, shows that NAND flash specifications primarily determine the performance of LLM inference in resource-constrained devices such as smartphones. NAND flash with larger capacity, which includes more planes, results in higher performance by increasing the plane-level parallelism of Flash-PIM. Fig. 16(a) shows that TPOT of the proposed architecture improves with larger capacities. For instance, a 1024GB NAND flash, the largest capacity available in commercial smartphones, achieves a $3.4\times$ speedup over a 128GB NAND flash, the smallest available

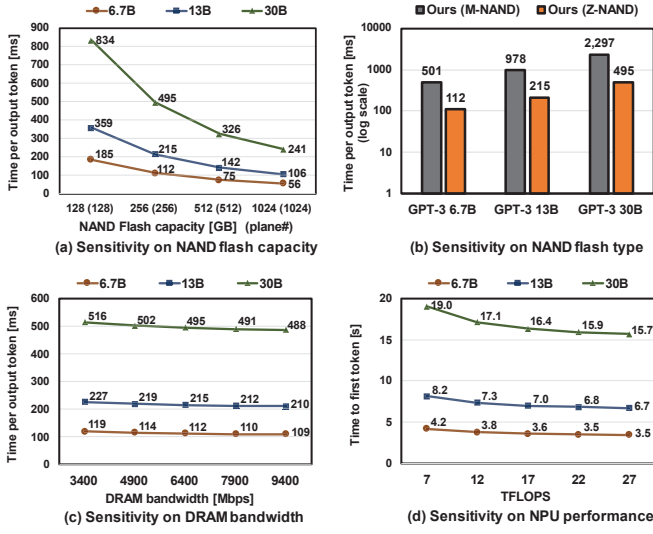


Fig. 16: Sensitivity analysis of the proposed architecture on (a) NAND flash capacities, (b) types of NAND flash, (c) DRAM bandwidth, and (d) NPU performance.

capacity. However, this speedup scales sub-linearly with capacity because the shared channel incurs increases command transfer delays as the number of planes increases.

The type of NAND flash also has a substantial impact on performance. We evaluate the TPOT of the proposed architecture using medium-latency NAND flash [9], referred to as M-NAND in this paper, and compare it with our previous evaluations with low-latency NAND Flash (Z-NAND) [10]. Fig. 16(b) shows that Z-NAND delivers a 4.6 $\times$ speedup over M-NAND. Although Z-NAND offers both high throughput and low latency for cell reads, conventional memory usage cannot fully exploit the throughput due to channel bandwidth bottlenecks, limiting its advantage to lower latency only. On the other hand, Flash-PIM bypasses the channel bandwidth bottleneck by computing before data is transferred. This enables full utilization of the high cell read throughput of Z-NAND, resulting in higher performance.

For LLM inference on mobile systems with limited DRAM capacity, the latency of NAND flash dominates the overall LLM performance, making it much more important than DRAM bandwidth and NPU compute throughput. Fig. 16(c) shows that increasing DRAM bandwidth from 3400Mbps to 9400Mbps results in only a 1.1 $\times$ geomean speedup. Similarly, Fig. 16(d) shows that increasing NPU throughput from 7 TFLOPS to 27 TFLOPS achieves just a 1.2 $\times$ speedup.

VI. DISCUSSION

A. Exclusion of error correction

The proposed Flash-PIM architecture leverages single-level cell (SLC) NAND flash, which offers a sufficiently low bit error rate (BER) ensuring reliable computation and storage without error correction code (ECC). SLC is well-suited for Flash-PIM as it provides low cell read latency enabling high-throughput Flash-PIM operations. This design choice aligns with prior works such as [25] and [5]. In contrast, multi-level cell (MLC) NAND flash exhibits significantly higher BER due to reduced read margins, necessitating ECC for data integrity. However, the conventional ECC module locates at SSD controller, requiring off-chip data transfer for error correction. To support Flash-PIM in MLC-based systems, we can additionally adopt

the approach of Cambricon-LLM [6] that integrates an on-die ECC module within each plane consuming additional latency and area.

B. Comparison to in-storage processing

In-storage processing (ISP), which integrates compute logic into the controller [26]–[28], improves the performance over conventional NPU-only systems for LLM inference by offloading weight computation to storage. ISP typically can employ more complex compute units within the controller die. However, ISP must fetch large weight matrices from NAND flash to the controller over off-chip interfaces with limited bandwidth (e.g., 3.2GB/s per channel [16]). This introduces significant latency and energy overhead, especially for memory-bound workloads including GEMV in LLM decoding. Meanwhile, our Flash-PIM architecture processes data directly in NAND flash with much higher bandwidth (e.g., 117GB/s for 256GB NAND flash), achieving 4.3 $\times$ speedup over ISP that assumes the NPU integrated in the flash controller under our system configuration.

C. Other applications

The proposed architecture can support a wide range of applications that includes MAC computation, which serve as fundamental computation kernels in various domains. In vector databases, which store text, image, or audio as high-dimensional vectors, dot product operations play a critical role in measuring similarity between stored vectors and input query vector [29]–[31]. By storing these large-scale vectors in NAND flash, our Flash-PIM-based MAC engine efficiently accelerates the similarity computation, benefiting key applications such as Retrieval-Augmented Generation (RAG) [32], recommendation systems [33], and semantic search [34]. The architecture also applies to signal processing workloads like the Discrete Fourier Transform (DFT) [35]–[37]. DFT decomposes discrete signals into their constituent frequency components through MAC computations. Applications such as speech recognition, music analysis, and image processing, heavily rely on DFT.

VII. CONCLUSION

We propose a Flash-PIM-based architecture that enables efficient LLM inference on smartphones, which has limited DRAM capacity. Conventional mobile systems suffer from excessive data transfer overhead, as it must repeatedly load parameters from NAND flash during inference. The proposed architecture addresses this bottleneck by embedding lightweight compute units in NAND flash die, eliminating the off-chip weight transfer. Unlike previous Flash-PIM-based architecture, our design fully exploits the high cell read bandwidth of NAND flash by placing compute units in every plane. As a result, the proposed architecture achieves a 3.7 $\times$ speedup and a 4.5 $\times$ reduction in energy consumption compared to the previous architecture, offering an efficient and practical solution for on-device LLM inference.

ACKNOWLEDGMENT

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2023-00208606, NeuroHub+: Scheduler and Simulator for General In-Memory Neural Network Accelerators, RS-2022-II220266, Development of Ultra-Low Power Low-Bit Precision Mixed-mode SRAM PIM, No. 2021-0-01343: Artificial Intelligence Graduate School Program (Seoul National University)), and BK21 FOUR program at Seoul National University. (Corresponding Author: Jae-Joon Kim).

REFERENCES

- [1] Badhan Chandra Das, M Hadi Amini, and Yanzhao Wu. Security and privacy challenges of large language models: A survey. *ACM Computing Surveys*, 57(6):1–39, 2025.
- [2] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. Mobile edge intelligence for large language models: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 2025.
- [3] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. Llm in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12562–12584, 2024.
- [4] Marc Brysbaert. How many words do we read per minute? a review and meta-analysis of reading rate. *Journal of memory and language*, 109:104047, 2019.
- [5] Junkyum Kim, Myeonggu Kang, Yunki Han, Yang-Gon Kim, and Lee-Sup Kim. Optimstore: In-storage optimization of large scale dnns with on-die processing. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 611–623. IEEE, 2023.
- [6] Zhongkai Yu, Shengwen Liang, Tianyun Ma, Yunke Cai, Ziyuan Nan, Di Huang, Xinkai Song, Yifan Hao, Jie Zhang, Tian Zhi, et al. Cambricon-llm: A chiplet-based hybrid architecture for on-device inference of 70b llm. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1474–1488. IEEE, 2024.
- [7] Myoungjun Chun, Jaeyong Lee, Myungsuk Kim, Jisung Park, and Jihong Kim. Rif: Improving read performance of modern ssds using an on-die early-retry engine. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 643–656. IEEE, 2024.
- [8] Congming Gao, Liang Shi, Chun Jason Xue, Cheng Ji, Jun Yang, and Youtao Zhang. Parallel all the time: Plane level parallelism exploration for high performance ssds. In *2019 35th Symposium on Mass Storage Systems and Technologies (MSST)*, pages 172–184. IEEE, 2019.
- [9] Wontaek Jung, Hyunggon Kim, Do-Bin Kim, Tae-Hyun Kim, Namhee Lee, Dongjin Shin, Minyoung Kim, Youngsik Rho, Hun-Jong Lee, Yujin Hyun, et al. 13.3 a 280-layer 1tb 4b/cell 3d-nand flash memory with a 28.5 gb/mm² areal density and a 3.2 gb/s high-speed io rate. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 67, pages 236–237. IEEE, 2024.
- [10] Woosung Cheong, Chanho Yoon, Seonghoon Woo, Kyuwook Han, Daehyun Kim, Chulseung Lee, Youra Choi, Shine Kim, Dongku Kang, Geunyeong Yu, et al. A flash memory controller for 15 μ s ultra-low-latency ssd using high-speed 3d nand flash with 3 μ s read time. In *2018 IEEE International Solid-State Circuits Conference-(ISSCC)*, pages 338–340. IEEE, 2018.
- [11] Sara Hooker, Nyalleng Moorosi, Gregory Clark, Samy Bengio, and Emily Denton. Characterising bias in compressed models. *arXiv preprint arXiv:2010.03058*, 2020.
- [12] Andrey Kuzmin, Markus Nagel, Mart Van Baalen, Arash Behboodi, and Tijmen Blankevoort. Pruning vs quantization: Which is better? *Advances in neural information processing systems*, 36:62414–62427, 2023.
- [13] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.
- [14] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelu). *arXiv preprint arXiv:1606.08415*, 2016.
- [15] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [16] Wontaek Jung, Hyunggon Kim, Do-Bin Kim, Tae-Hyun Kim, Namhee Lee, Dongjin Shin, Minyoung Kim, Youngsik Rho, Hun-Jong Lee, Yujin Hyun, et al. 13.3 a 280-layer 1tb 4b/cell 3d-nand flash memory with a 28.5 gb/mm² areal density and a 3.2 gb/s high-speed io rate. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 67, pages 236–237. IEEE, 2024.
- [17] Open NAND Flash Interface (ONFI). Open nand flash interface specification revision 5.1, 2022. <https://www.onfi.org/>.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [19] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–624, 2023.
- [20] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- [21] Samsung. Ufs 4.0 : Flagship storage, 2023.
- [22] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. {MQSim}: A framework for enabling realistic studies of modern {Multi-Queue}{SSD} devices. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*, pages 49–66, 2018.
- [23] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. Dramsim3: A cycle-accurate, thermal-capable dram simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.
- [24] SYNOPTICS. Design compiler, 2018.
- [25] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. Flash-cosmos: In-flash bulk bitwise operations using inherent computation capability of nand flash memory. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 937–955. IEEE, 2022.
- [26] Boncheol Gu, Andre S Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, et al. Biscuit: A framework for near-data processing of big data workloads. *ACM SIGARCH Computer Architecture News*, 44(3):153–165, 2016.
- [27] Chen Zou and Andrew A Chien. Assassin: Architecture support for stream computing to accelerate computational storage. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 354–368. IEEE, 2022.
- [28] Shine Kim, Yunho Jin, Gina Sohn, Jonghyun Bae, Tae Jun Ham, and Jae W Lee. Behemoth: a flash-centric training accelerator for extreme-scale {DNNs}. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 371–385, 2021.
- [29] James Jie Pan, Jianguo Wang, and Guoliang Li. Survey of vector database management systems. *The VLDB Journal*, 33(5):1591–1615, 2024.
- [30] James Jie Pan, Jianguo Wang, and Guoliang Li. Vector database management techniques and systems. In *Companion of the 2024 International Conference on Management of Data*, pages 597–604, 2024.
- [31] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. Singlestore-v: An integrated vector database system in single-store. *Proceedings of the VLDB Endowment*, 17(12):3772–3785, 2024.
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [33] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, and Qing Li. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*, 36(11):6889–6907, November 2024.
- [34] Ramanathan Guha, Rob McCool, and Eric Miller. Semantic search. In *Proceedings of the 12th international conference on World Wide Web*, pages 700–709, 2003.
- [35] Duraisamy Sundarajan. *The discrete Fourier transform: theory, algorithms and applications*. World Scientific, 2001.
- [36] Tim Olson and Tim Olson. The discrete fourier transform. *Applied Fourier Analysis: From Signal Processing to Medical Imaging*, pages 75–120, 2017.
- [37] Tiziano Bianchi, Alessandro Piva, and Mauro Barni. On the implementation of the discrete fourier transform in the encrypted domain. *IEEE Transactions on Information Forensics and Security*, 4(1):86–97, 2009.